

Intermediate (build real memory) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `conversation_memory_practice.py`: an LLM API call has no memory of its own — every call is stateless. "Memory" is really just resending the whole conversation so far, every single time. Building that resend-everything loop by hand, once, here, is what makes the Build Task's `chat_client.py` obvious instead of magic. **If not:** you'd be handed `send_message(history, user_input)` in the Build Task without ever having seen why it needs the whole `history` list passed in every time, not just the newest message.

All examples below assume `client = OpenAI()` with `.env` already loaded.

Basic Version

Approach 1 — the direct way

```
# conversation_memory_practice.py
history = [
    {"role": "system", "content": "You are a helpful assistant."}
]

while True:
    user_input = input("You: ")
    if user_input == "quit":
        break

    history.append({"role": "user", "content": user_input})

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
    )
    reply = response.choices[0].message.content

    history.append({"role": "assistant", "content": reply})
    print("Assistant:", reply)
```

Expected output (a real run):

You: My favorite color is teal.

Assistant: Teal is a great choice! It's a lovely blend of blue and green...

You: What's my favorite color?

Assistant: You mentioned your favorite color is teal!

This works correctly and proves memory. It has no error handling, and it never limits how large `history` can grow, which matters once conversations get long.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — `get_reply()` pulled out, with type hints

```
# conversation_memory_practice.py
def get_reply(history: list) -> str:
    # why: this exact shape – take the whole history, return just the new
    # reply text – is what the Build Task's send_message() needs to look like.
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
    )
    return response.choices[0].message.content

def run_chat() -> None:
    history = [
        {"role": "system", "content": "You are a helpful assistant."}
    ]

    while True:
        user_input = input("You: ")
        if user_input == "quit":
            break

        # how: the model only ever sees history – it has no memory of its
        # own, so every earlier turn has to be resent, every single time.
        history.append({"role": "user", "content": user_input})

        reply = get_reply(history)
        history.append({"role": "assistant", "content": reply})

        print("Assistant:", reply)

if __name__ == "__main__":
    run_chat()
```

Expected output: identical conversation to Basic — same behavior, cleaner structure.

Difference from Basic: pulling the API call out into `get_reply(history)` separates "how to talk to the model" from "how the loop is structured" — this is the exact shape the Build Task's `send_message()` needs to take. Full type hints on `get_reply` document that it takes the whole history list and returns just the new reply text, not a whole response object the caller has to unpack.

Which one should you actually write? For a single-user terminal script — which is exactly what this document's exercises and Build Task are — Intermediate's plain `history` list, held in a loop, is completely fine, and simpler to read. A class that owns `history` as instance state is worth reaching for the moment more than one conversation genuinely needs to exist at the same time (a web server handling multiple users) — not needed yet here.